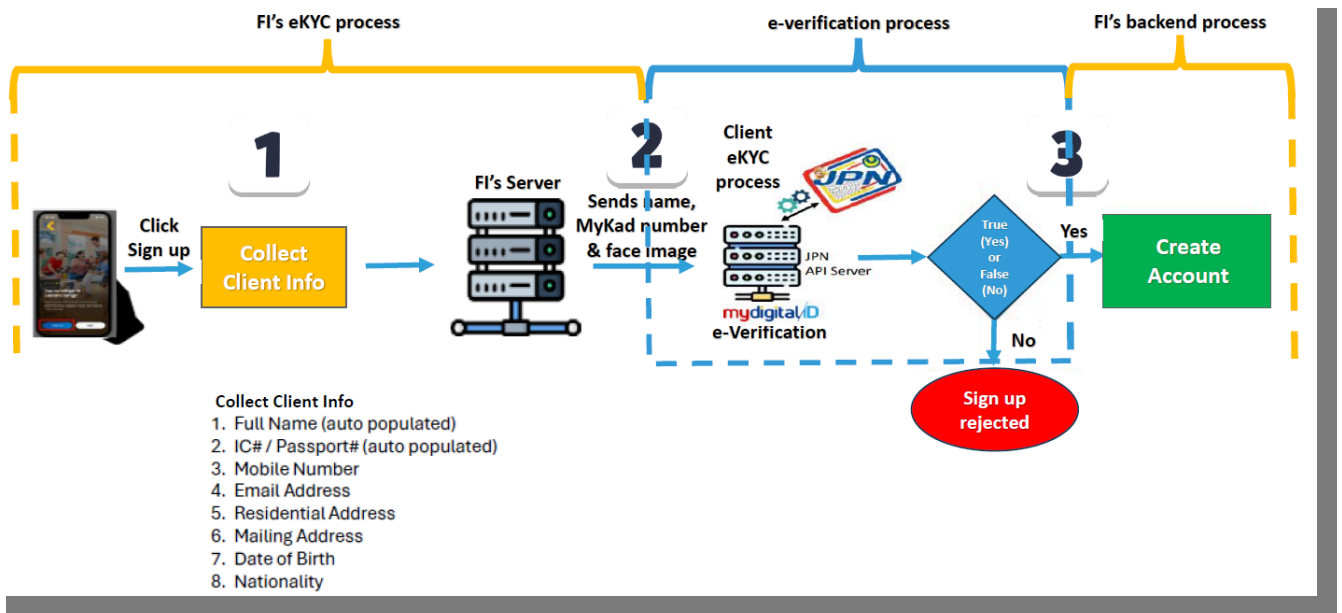


MyID Verify V1 Integration Guide

Integrating with **MyID Verify (MyDigital ID Static E-Verification)** allows your system to verify a user's identity by comparing their submitted details (Name, IC Number, and Face Image) directly against the Jabatan Pendaftaran Negara (JPN) national database using facial recognition scoring.

Unlike Live E-Verification, Static E-Verification does not perform real-time liveness or anti-spoofing checks; it focuses on image comparison and hash matching against official records.



High-Level Integration Workflow

[Your Current EKYC Module Frontend / Mobile App]



1. Collect User Info (Name, IC Number) & Capture Face Image



[Your Application Server]



2. Call Static E-Verification API (multipart/form-data)



[MyDigital ID E-Verification API Gateway]



3. Validate Name & IC against JPN records

4. Perform face comparison with JPN image



[Response returned to Your Application Server]

(Status, Session ID, Face Match Boolean, Face Score 0-100)

Key Technical Details

1. API Endpoint & Requirements

- **API URL:** [HOST]/ekyc/face-verify/face-scoring-from-id
- **HTTP Method:** POST
- **Content-Type:** multipart/form-data
- **Authentication:** Bearer [API_TOKEN]

2. Request Body Parameters

Parameter	Data Type	Required	Description
name	Text	Yes	Valid full name of the user as printed on their MyKad.
icNumber	Text	Yes	Valid IC Number (can be provided with or without hyphens).
imageFile	File	Yes	Clear face photo file (JPEG/PNG with datasize below 7mb) captured or uploaded during user registration.

API Response Examples

Match Successful Response (faceMatch: true)

```
{
  "status": "success",
  "code": "SUCCESS",
  "subcode": 0,
  "sessionId": "65a24536-20bf-4464-a6c8-2e6e6ed660b9",
  "data": {
    "faceMatch": true,
    "faceScore": 100.0
  },
  "meta": {
    "reqTs": 1751959401469,
    "respTs": 1751959402575,
    "reqId": "32718d42-89f9-45af-8cd2-1ae48635f6bb"
  },
  "hashData": "07A4D24E58DC490BDD932C0DB6D8B753988B1F13E0C7507..."
}
```

Match Failed Response (faceMatch: false)

```
{
  "status": "success",
  "code": "SUCCESS",
  "subcode": 0,
  "sessionId": "65a24536-20bf-4464-a6c8-2e6e6ed660b9",
  "data": {
    "faceMatch": false,
    "faceScore": 65.7
  },
  "meta": {
    "reqTs": 1751959401469,
    "respTs": 1751959402575,
    "reqId": "32718d42-89f9-45af-8cd2-1ae48635f6bb"
  }
}
```

Implementation Steps

1. **Obtain API Credentials:** Acquire your API host URL and Bearer Token from the MyDigital ID Partners Portal.
2. **User Input Collection:** Capture the user's Full Name, MyKad IC Number, and a clear face photo on your application frontend.
3. **Backend API Request:** Send a POST request from your backend server to the MyDigital ID API using multipart/form-data.
4. **Process Verification Result:**
 - Inspect data.faceMatch (true or false) and data.faceScore in the response.
 - If faceMatch is true, proceed with account creation or identity verification.
 - If faceMatch is false, flag the verification for manual review or prompt the user to re-capture their photo.

This section describes static e-Verification API:

Description	Verify the face image with JPN to get the face scoring.
API Link	[HOST]/ekyc/face-verify/face-scoring-from-id
Authorization	Bearer [API_TOKEN]
Prerequisite	Client will need to have the face image
Content-Type	multipart/form-data

Body:

Key	Data Type	Required	Description
name	Text	Yes	User's valid name as per ICnumber
icNumber	Text	Yes	User's valid IC number, with or without '-'
imageFile	File	Yes	The face image file

Condition	Response
When successfully get response, and the face image matches with face in JPN	<pre>{ "status": "success", "code": "SUCCESS", "subcode": 0, "sessionId": "65a24536-20bf-4464-a6c8-2e6e6ed660b9", "data": { "faceMatch": true, "faceScore": 100.0 }, "meta": { "reqTs": 1751959401469, "respTs": 1751959402575, "reqId": "32718d42-89f9-45af-8cd2-1ae48635f6bb" }, "hashData":</pre>

Condition	Response
	<pre> "07A4D24E58DC490BDD932C0DB6D8B753988B1F13E0C75 07 } </pre>
When the face image does not match with face in JPN	<pre> { "status": "success", "code": "SUCCESS", "subcode": 0, "sessionId": "65a24536-20bf-4464-a6c8-2e6e6ed660b9", "data": { "faceMatch": false, "faceScore": 65.7 }, "meta": { "reqTs": 1751959401469, "respTs": 1751959402575, "reqId": "32718d42-89f9-45af-8cd2-1ae48635f6bb" }, "hashData": "07A4D24E58DC490BDD932C0DB6D8B753988B1F13E 0C7507 } </pre>
When name is not provided	<pre> { "status": "error", "code": "INVALID_INPUT", "subcode": 90, "message": "Name is required", "meta": { "reqTs": 1751873664570, "respTs": 1751873664666, "reqId": "7c0d7431-c63d-481b-b85a-ae9c11210d8d" } } </pre>

Condition	Response
	<pre> } } </pre>
When IC number is not provided	<pre> { "status": "error", "code": "INVALID_INPUT", "subcode":90, "message": "IC number is required", "meta": { "reqTs":1751873702593, "respTs":1751873702711, "reqId": "85566a4d-4c1e-4904-8a01-b41d3f4803b6" } } </pre>
When face image is not provided	<pre> { "status": "error", "code": "INVALID_INPUT", "subcode":90, "message": "Image file is required", "meta": { "reqTs":1751873719401, "respTs":1751873719540, "reqId": "ab4bdffc-0d3a-418d-9fb4-e534de038cd0" } } </pre>
When IC format is invalid	<pre> { "status": "error", "code": "INVALID_IC_NUMBER", "subcode":90, "message": "Invalid IC number format", "meta": { "reqTs":1751873719401, </pre>

Condition	Response
	<pre> "respTs":1751873719540, "reqId": "ab4bdffc-0d3a-418d-9fb4-e534de038cd0" } </pre>
When invalid API Key is used	<pre> { "status": "error", "code": "UNAUTHORIZED", "subcode":90, "message": "Invalid token", "meta": { "reqTs":1751937691424, "respTs":1751937691432, "reqId": "cf6d7f7b-a39e-4ca4-9ea0-b5d191613224" } } </pre>
When image file format is not supported	<pre> { "status": "error", "code": "INVALID_INPUT", "subcode":90, "message": "File format of image file is not supported", "meta": { "reqTs":1751873719401, "respTs":1751873719540, "reqId": "ab4bdffc-0d3a-418d-9fb4-e534de038cd0" } } </pre>
When image file size exceeds 7 MB	<pre> { "status": "error", "code": "INVALID_INPUT", "subcode":90, "message": "File size of image file exceeds the maximum supported </pre>

Condition	Response
	<pre>size (7 "meta": { "reqTs":1751873719401, "respTs":1751873719540, "reqId": "ab4bdffc-0d3a-418d-9fb4-e534de038cd0" } }</pre>
When IC and name hash doesn't not match with JPN	<pre>{ "status": "error", "code": "VERIFY_HASH_FAILED", "subcode":101, "message": "Verify hash not match", "sessionId": "a370c720-c49e-441c-9877-31447911f077", "meta": { "reqTs":1751963936778, "respTs":1751963937469, "reqId": "34f9e193-7258-4626-befe-3932786cdca7" }, "hashData": "74234E98AFE7498FB5DAF1F36AC2D78ACC339464F950703" }</pre>
When connection to JPN failed on verify hashing	<pre>{ "status": "error", "code": "VERIFY_HASH_FAILED", "subcode":101, "message": "The connection for JPN Verify Hash was not successful", "sessionId": "072aa543-1f61-45f4-9735-7eeb0d669758", "meta": { "reqTs":1751873561048, "respTs":1751873561948, "reqId": "019a6ae9-7ee4-459f-a8e1-8182fd4c4c" } }</pre>

Condition	Response
	<pre> }, "hashData": "74234E98AFE7498FB5DAF1F36AC2D78ACC339464F950703 } </pre>
When verify hash failed due to other reasons	<pre> { "status": "error", "code": "VERIFY_HASH_FAILED", "subcode":101, "message": "Verify Hash Failed Due to Unexpected Reasons", "sessionId": "072aa543-1f61-45f4-9735-7eeb0d669758", "meta": { "reqTs":1751873561048, "respTs":1751873561948, "reqId": "019a6ae9-7ee4-459f-a8e1-8182fdfd4c4c" }, "hashData": "74234E98AFE7498FB5DAF1F36AC2D78ACC339464F950703 } </pre>
When no face is found	<pre> { "status": "error", "code": "FACE_DETECT_ERROR", "subcode":82, "message": "No face detected within the image or video", "sessionId": "072aa543-1f61-45f4-9735-7eeb0d669758", "meta": { "reqTs":1751873561048, "respTs":1751873561948, "reqId": "019a6ae9-7ee4-459f-a8e1-8182fdfd4c4c" }, "hashData": "74234E98AFE7498FB5DAF1F36AC2D78ACC339464F950703 } </pre>

Condition	Response
When connection to JPN failed on verify feature vector	<pre>{ "status": "error", "code": "VERIFY_FEATURE_VECTOR_FAILED", "subcode":102, "message": "The connection for JPN Verify Face with feature vector was no "sessionId": "072aa543-1f61-45f4-9735-7eeb0d669758", "meta": { "reqTs":1751873561048, "respTs":1751873561948, "reqId": "019a6ae9-7ee4-459f-a8e1-8182fd4c4c" }, "hashData": "74234E98AFE7498FB5DAF1F36AC2D78ACC339464F950703 }</pre>
When verify feature vector failed due to other reasons	<pre>{ "status": "error", "code": "VERIFY_FEATURE_VECTOR_FAILED", "subcode":102, "message": "Verify Feature Vector Failed Due to Unexpected Reasons", "sessionId": "072aa543-1f61-45f4-9735-7eeb0d669758", "meta": { "reqTs":1751873561048, "respTs":1751873561948, "reqId": "019a6ae9-7ee4-459f-a8e1-8182fd4c4c" }, "hashData": "74234E98AFE7498FB5DAF1F36AC2D78ACC339464F950703 }</pre>
When unexpected error	<pre>{ "status": "error",</pre>

Condition	Response
occurred	<pre>"code": "INTERNAL_SERVER_ERROR", "subcode":99, "message": "Internal server error", "meta": { "reqTs":1750906190377, "respTs":1750906220560, "reqId": "d1ecf122-fb81-44dc-ad99-9a83272157c5" } }</pre>